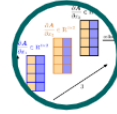
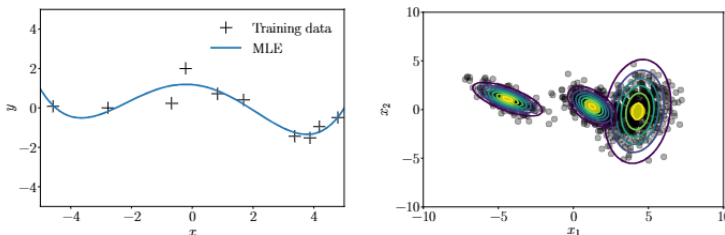


Vector Calculus



Many algorithms in machine learning optimize an objective function with respect to a set of desired model parameters that control how well a model explains the data: Finding good parameters can be phrased as an optimization problem (see Sections 8.2 and 8.3). Examples include: (i) linear regression (see Chapter 9), where we look at curve-fitting problems and optimize linear weight parameters to maximize the likelihood; (ii) neural-network auto-encoders for dimensionality reduction and data compression, where the parameters are the weights and biases of each layer, and where we minimize a reconstruction error by repeated application of the chain rule; and (iii) Gaussian mixture models (see Chapter 11) for modeling data distributions, where we optimize the location and shape parameters of each mixture component to maximize the likelihood of the model. Figure 5.1 illustrates some of these problems, which we typically solve by using optimization algorithms that exploit gradient information (Section 7.1). Figure 5.2 gives an overview of how concepts in this chapter are related and how they are connected to other chapters of the book.

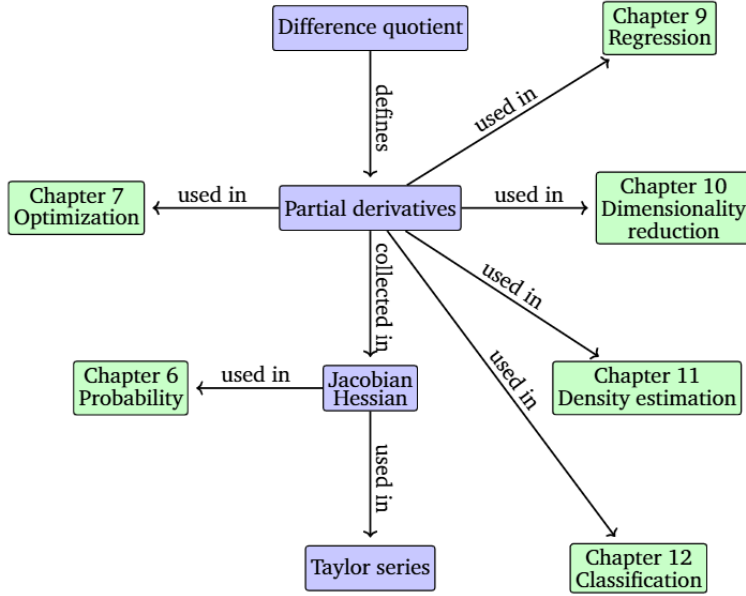
Central to this chapter is the concept of a function. A function f is a quantity that relates two quantities to each other. In this book, these quantities are typically inputs $x \in \mathbb{R}^D$ and targets (function values) $f(x)$, which we assume are real-valued if not stated otherwise. Here $\mathbb{R}^D$ is the *domain* of f , and the function values $f(x)$ are the *image/codomain* of f .



(a) Regression problem: Find parameters, such that the curve explains the observations (crosses) well. (b) Density estimation with a Gaussian mixture model: Find means and covariances, such that the data (dots) can be explained well.

domain
image/codomain
Figure 5.1 Vector calculus plays a central role in (a) regression (curve fitting) and (b) density estimation, i.e., modeling data distributions.

Figure 5.2 A mind map of the concepts introduced in this chapter, along with when they are used in other parts of the book.



Section 2.7.3 provides much more detailed discussion in the context of linear functions. We often write

$$f : \mathbb{R}^D \rightarrow \mathbb{R} \quad (5.1a)$$

$$x \mapsto f(x) \quad (5.1b)$$

to specify a function, where (5.1a) specifies that f is a mapping from $\mathbb{R}^D$ to $\mathbb{R}$ and (5.1b) specifies the explicit assignment of an input x to a function value $f(x)$. A function f assigns every input x exactly one function value $f(x)$.

Example 5.1

Recall the dot product as a special case of an inner product (Section 3.2). In the previous notation, the function $f(x) = x^\top x$, $x \in \mathbb{R}^2$, would be specified as

$$f : \mathbb{R}^2 \rightarrow \mathbb{R} \quad (5.2a)$$

$$x \mapsto x_1^2 + x_2^2. \quad (5.2b)$$

In this chapter, we will discuss how to compute gradients of functions, which is often essential to facilitate learning in machine learning models since the gradient points in the direction of steepest ascent. Therefore,

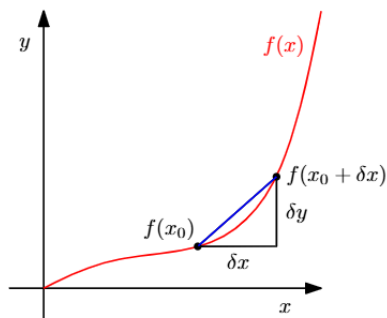


Figure 5.3 The average incline of a function f between x_0 and $x_0 + \delta x$ is the incline of the secant (blue) through $f(x_0)$ and $f(x_0 + \delta x)$ and given by $\delta y / \delta x$.

vector calculus is one of the fundamental mathematical tools we need in machine learning. Throughout this book, we assume that functions are differentiable. With some additional technical definitions, which we do not cover here, many of the approaches presented can be extended to sub-differentials (functions that are continuous but not differentiable at certain points). We will look at an extension to the case of functions with constraints in Chapter 7.

5.1 Differentiation of Univariate Functions

In the following, we briefly revisit differentiation of a univariate function, which may be familiar from high school mathematics. We start with the difference quotient of a univariate function $y = f(x)$, $x, y \in \mathbb{R}$, which we will subsequently use to define derivatives.

Definition 5.1 (Difference Quotient). The *difference quotient*

difference quotient

$$\frac{\delta y}{\delta x} := \frac{f(x + \delta x) - f(x)}{\delta x} \quad (5.3)$$

computes the slope of the secant line through two points on the graph of f . In Figure 5.3, these are the points with x -coordinates x_0 and $x_0 + \delta x$.

The difference quotient can also be considered the average slope of f between x and $x + \delta x$ if we assume f to be a linear function. In the limit for $\delta x \rightarrow 0$, we obtain the tangent of f at x , if f is differentiable. The tangent is then the derivative of f at x .

Definition 5.2 (Derivative). More formally, for $h > 0$ the *derivative* of f at x is defined as the limit

derivative

$$\frac{df}{dx} := \lim_{h \rightarrow 0} \frac{f(x + h) - f(x)}{h}, \quad (5.4)$$

and the secant in Figure 5.3 becomes a tangent.

The derivative of f points in the direction of steepest ascent of f .

Example 5.2 (Derivative of a Polynomial)

We want to compute the derivative of $f(x) = x^n$, $n \in \mathbb{N}$. We may already know that the answer will be nx^{n-1} , but we want to derive this result using the definition of the derivative as the limit of the difference quotient.

Using the definition of the derivative in (5.4), we obtain

$$\frac{df}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \quad (5.5a)$$

$$= \lim_{h \rightarrow 0} \frac{(x+h)^n - x^n}{h} \quad (5.5b)$$

$$= \lim_{h \rightarrow 0} \frac{\sum_{i=0}^n \binom{n}{i} x^{n-i} h^i - x^n}{h}. \quad (5.5c)$$

We see that $x^n = \binom{n}{0} x^{n-0} h^0$. By starting the sum at 1, the x^n -term cancels, and we obtain

$$\frac{df}{dx} = \lim_{h \rightarrow 0} \frac{\sum_{i=1}^n \binom{n}{i} x^{n-i} h^i}{h} \quad (5.6a)$$

$$= \lim_{h \rightarrow 0} \sum_{i=1}^n \binom{n}{i} x^{n-i} h^{i-1} \quad (5.6b)$$

$$= \lim_{h \rightarrow 0} \left(\binom{n}{1} x^{n-1} + \underbrace{\sum_{i=2}^n \binom{n}{i} x^{n-i} h^{i-1}}_{\rightarrow 0 \text{ as } h \rightarrow 0} \right) \quad (5.6c)$$

$$= \frac{n!}{1!(n-1)!} x^{n-1} = nx^{n-1}. \quad (5.6d)$$

5.1.1 Taylor Series

The Taylor series is a representation of a function f as an infinite sum of terms. These terms are determined using derivatives of f evaluated at x_0 .

Definition 5.3 (Taylor Polynomial). The *Taylor polynomial* of degree n of $f : \mathbb{R} \rightarrow \mathbb{R}$ at x_0 is defined as

$$T_n(x) := \sum_{k=0}^n \frac{f^{(k)}(x_0)}{k!} (x - x_0)^k, \quad (5.7)$$

where $f^{(k)}(x_0)$ is the k th derivative of f at x_0 (which we assume exists) and $\frac{f^{(k)}(x_0)}{k!}$ are the coefficients of the polynomial.

Definition 5.4 (Taylor Series). For a smooth function $f \in \mathcal{C}^\infty$, $f : \mathbb{R} \rightarrow \mathbb{R}$, the *Taylor series* of f at x_0 is defined as

Taylor polynomial
We define $t^0 := 1$
for all $t \in \mathbb{R}$.

Taylor series

$$T_{\infty}(x) = \sum_{k=0}^{\infty} \frac{f^{(k)}(x_0)}{k!} (x - x_0)^k. \quad (5.8)$$

For $x_0 = 0$, we obtain the *Maclaurin series* as a special instance of the Taylor series. If $f(x) = T_{\infty}(x)$, then f is called *analytic*.

$f \in C^{\infty}$ means that f is continuously differentiable infinitely many times. Maclaurin series analytic

Remark. In general, a Taylor polynomial of degree n is an approximation of a function, which does not need to be a polynomial. The Taylor polynomial is similar to f in a neighborhood around x_0 . However, a Taylor polynomial of degree n is an exact representation of a polynomial f of degree $k \leq n$ since all derivatives $f^{(i)}$, $i > k$ vanish. $\diamond$

Example 5.3 (Taylor Polynomial)

We consider the polynomial

$$f(x) = x^4 \quad (5.9)$$

and seek the Taylor polynomial T_6 , evaluated at $x_0 = 1$. We start by computing the coefficients $f^{(k)}(1)$ for $k = 0, \dots, 6$:

$$f(1) = 1 \quad (5.10)$$

$$f'(1) = 4 \quad (5.11)$$

$$f''(1) = 12 \quad (5.12)$$

$$f^{(3)}(1) = 24 \quad (5.13)$$

$$f^{(4)}(1) = 24 \quad (5.14)$$

$$f^{(5)}(1) = 0 \quad (5.15)$$

$$f^{(6)}(1) = 0 \quad (5.16)$$

Therefore, the desired Taylor polynomial is

$$T_6(x) = \sum_{k=0}^6 \frac{f^{(k)}(x_0)}{k!} (x - x_0)^k \quad (5.17a)$$

$$= 1 + 4(x - 1) + 6(x - 1)^2 + 4(x - 1)^3 + (x - 1)^4 + 0. \quad (5.17b)$$

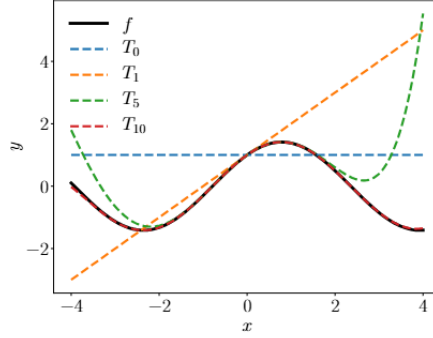
Multiplying out and re-arranging yields

$$T_6(x) = (1 - 4 + 6 - 4 + 1) + x(4 - 12 + 12 - 4) + x^2(6 - 12 + 6) + x^3(4 - 4) + x^4 \quad (5.18a)$$

$$= x^4 = f(x), \quad (5.18b)$$

i.e., we obtain an exact representation of the original function.

Figure 5.4 Taylor polynomials. The original function $f(x) = \sin(x) + \cos(x)$ (black, solid) is approximated by Taylor polynomials (dashed) around $x_0 = 0$. Higher-order Taylor polynomials approximate the function f better and more globally. T_{10} is already similar to f in $[-4, 4]$.



Example 5.4 (Taylor Series)

Consider the function in Figure 5.4 given by

$$f(x) = \sin(x) + \cos(x) \in \mathcal{C}^\infty. \quad (5.19)$$

We seek a Taylor series expansion of f at $x_0 = 0$, which is the Maclaurin series expansion of f . We obtain the following derivatives:

$$f(0) = \sin(0) + \cos(0) = 1 \quad (5.20)$$

$$f'(0) = \cos(0) - \sin(0) = 1 \quad (5.21)$$

$$f''(0) = -\sin(0) - \cos(0) = -1 \quad (5.22)$$

$$f^{(3)}(0) = -\cos(0) + \sin(0) = -1 \quad (5.23)$$

$$f^{(4)}(0) = \sin(0) + \cos(0) = f(0) = 1 \quad (5.24)$$

$\vdots$

We can see a pattern here: The coefficients in our Taylor series are only ± 1 (since $\sin(0) = 0$), each of which occurs twice before switching to the other one. Furthermore, $f^{(k+4)}(0) = f^{(k)}(0)$.

Therefore, the full Taylor series expansion of f at $x_0 = 0$ is given by

$$T_\infty(x) = \sum_{k=0}^{\infty} \frac{f^{(k)}(x_0)}{k!} (x - x_0)^k \quad (5.25a)$$

$$= 1 + x - \frac{1}{2!}x^2 - \frac{1}{3!}x^3 + \frac{1}{4!}x^4 + \frac{1}{5!}x^5 - \dots \quad (5.25b)$$

$$= 1 - \frac{1}{2!}x^2 + \frac{1}{4!}x^4 \mp \dots + x - \frac{1}{3!}x^3 + \frac{1}{5!}x^5 \mp \dots \quad (5.25c)$$

$$= \sum_{k=0}^{\infty} (-1)^k \frac{1}{(2k)!} x^{2k} + \sum_{k=0}^{\infty} (-1)^k \frac{1}{(2k+1)!} x^{2k+1} \quad (5.25d)$$

$$= \cos(x) + \sin(x), \quad (5.25e)$$

where we used the *power series representations*

$$\cos(x) = \sum_{k=0}^{\infty} (-1)^k \frac{1}{(2k)!} x^{2k}, \quad (5.26)$$

$$\sin(x) = \sum_{k=0}^{\infty} (-1)^k \frac{1}{(2k+1)!} x^{2k+1}. \quad (5.27)$$

power series
representation

Figure 5.4 shows the corresponding first Taylor polynomials T_n for $n = 0, 1, 5, 10$.

Remark. A Taylor series is a special case of a power series

$$f(x) = \sum_{k=0}^{\infty} a_k (x - c)^k \quad (5.28)$$

where a_k are coefficients and c is a constant, which has the special form in Definition 5.4. $\diamond$

5.1.2 Differentiation Rules

In the following, we briefly state basic differentiation rules, where we denote the derivative of f by f' .

Product rule: $(f(x)g(x))' = f'(x)g(x) + f(x)g'(x) \quad (5.29)$

Quotient rule: $\left(\frac{f(x)}{g(x)}\right)' = \frac{f'(x)g(x) - f(x)g'(x)}{(g(x))^2} \quad (5.30)$

Sum rule: $(f(x) + g(x))' = f'(x) + g'(x) \quad (5.31)$

Chain rule: $(g(f(x)))' = (g \circ f)'(x) = g'(f(x))f'(x) \quad (5.32)$

Here, $g \circ f$ denotes function composition $x \mapsto f(x) \mapsto g(f(x))$.

Example 5.5 (Chain Rule)

Let us compute the derivative of the function $h(x) = (2x + 1)^4$ using the chain rule. With

$$h(x) = (2x + 1)^4 = g(f(x)), \quad (5.33)$$

$$f(x) = 2x + 1, \quad (5.34)$$

$$g(f) = f^4, \quad (5.35)$$

we obtain the derivatives of f and g as

$$f'(x) = 2, \quad (5.36)$$

$$g'(f) = 4f^3, \quad (5.37)$$

such that the derivative of h is given as

$$h'(x) = g'(f)f'(x) = (4f^3) \cdot 2 \stackrel{(5.34)}{=} 4(2x+1)^3 \cdot 2 = 8(2x+1)^3, \quad (5.38)$$

where we used the chain rule (5.32) and substituted the definition of f in (5.34) in $g'(f)$.

5.2 Partial Differentiation and Gradients

Differentiation as discussed in Section 5.1 applies to functions f of a scalar variable $x \in \mathbb{R}$. In the following, we consider the general case where the function f depends on one or more variables $x \in \mathbb{R}^n$, e.g., $f(x) = f(x_1, x_2)$. The generalization of the derivative to functions of several variables is the *gradient*.

We find the gradient of the function f with respect to x by *varying one variable at a time* and keeping the others constant. The gradient is then the collection of these *partial derivatives*.

partial derivative

Definition 5.5 (Partial Derivative). For a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, $x \mapsto f(x)$, $x \in \mathbb{R}^n$ of n variables $x_1, \dots, x_n$ we define the *partial derivatives* as

$$\begin{aligned} \frac{\partial f}{\partial x_1} &= \lim_{h \rightarrow 0} \frac{f(x_1 + h, x_2, \dots, x_n) - f(x)}{h} \\ &\vdots \\ \frac{\partial f}{\partial x_n} &= \lim_{h \rightarrow 0} \frac{f(x_1, \dots, x_{n-1}, x_n + h) - f(x)}{h} \end{aligned} \quad (5.39)$$

and collect them in the row vector

$$\nabla_x f = \text{grad } f = \frac{df}{dx} = \begin{bmatrix} \frac{\partial f(x)}{\partial x_1} & \frac{\partial f(x)}{\partial x_2} & \dots & \frac{\partial f(x)}{\partial x_n} \end{bmatrix} \in \mathbb{R}^{1 \times n}, \quad (5.40)$$

where n is the number of variables and 1 is the dimension of the image/range/codomain of f . Here, we defined the column vector $x = [x_1, \dots, x_n]^T \in \mathbb{R}^n$. The row vector in (5.40) is called the *gradient* of f or the *Jacobian* and is the generalization of the derivative from Section 5.1.

gradient

Jacobian

Remark. This definition of the Jacobian is a special case of the general definition of the Jacobian for vector-valued functions as the collection of partial derivatives. We will get back to this in Section 5.3. $\diamond$

We can use results from scalar differentiation: Each partial derivative is a derivative with respect to a scalar.

Example 5.6 (Partial Derivatives Using the Chain Rule)

For $f(x, y) = (x + 2y^3)^2$, we obtain the partial derivatives

$$\frac{\partial f(x, y)}{\partial x} = 2(x + 2y^3) \frac{\partial}{\partial x}(x + 2y^3) = 2(x + 2y^3), \quad (5.41)$$

$$\frac{\partial f(x, y)}{\partial y} = 2(x + 2y^3) \frac{\partial}{\partial y}(x + 2y^3) = 12(x + 2y^3)y^2. \quad (5.42)$$

where we used the chain rule (5.32) to compute the partial derivatives.

Remark (Gradient as a Row Vector). It is not uncommon in the literature to define the gradient vector as a column vector, following the convention that vectors are generally column vectors. The reason why we define the gradient vector as a row vector is twofold: First, we can consistently generalize the gradient to vector-valued functions $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ (then the gradient becomes a matrix). Second, we can immediately apply the multi-variate chain rule without paying attention to the dimension of the gradient. We will discuss both points in Section 5.3. $\diamond$

Example 5.7 (Gradient)

For $f(x_1, x_2) = x_1^2 x_2 + x_1 x_2^3 \in \mathbb{R}$, the partial derivatives (i.e., the derivatives of f with respect to x_1 and x_2) are

$$\frac{\partial f(x_1, x_2)}{\partial x_1} = 2x_1 x_2 + x_2^3 \quad (5.43)$$

$$\frac{\partial f(x_1, x_2)}{\partial x_2} = x_1^2 + 3x_1 x_2^2 \quad (5.44)$$

and the gradient is then

$$\frac{df}{dx} = \left[\frac{\partial f(x_1, x_2)}{\partial x_1} \quad \frac{\partial f(x_1, x_2)}{\partial x_2} \right] = [2x_1 x_2 + x_2^3 \quad x_1^2 + 3x_1 x_2^2] \in \mathbb{R}^{1 \times 2}. \quad (5.45)$$

5.2.1 Basic Rules of Partial Differentiation

In the multivariate case, where $x \in \mathbb{R}^n$, the basic differentiation rules that we know from school (e.g., sum rule, product rule, chain rule; see also Section 5.1.2) still apply. However, when we compute derivatives with respect to vectors $x \in \mathbb{R}^n$ we need to pay attention: Our gradients now involve vectors and matrices, and matrix multiplication is not commutative (Section 2.2.1), i.e., the order matters.

Here are the general product rule, sum rule, and chain rule:

$$\text{Product rule:} \quad \frac{\partial}{\partial x}(f(x)g(x)) = \frac{\partial f}{\partial x}g(x) + f(x)\frac{\partial g}{\partial x} \quad (5.46)$$

$$\text{Sum rule:} \quad \frac{\partial}{\partial x}(f(x) + g(x)) = \frac{\partial f}{\partial x} + \frac{\partial g}{\partial x} \quad (5.47)$$

Product rule:

$$(fg)' = f'g + fg'$$

Sum rule:

$$(f + g)' = f' + g'$$

Chain rule:

$$(g(f))' = g'(f)f'$$

$$\text{Chain rule: } \frac{\partial}{\partial \mathbf{x}}(g \circ f)(\mathbf{x}) = \frac{\partial}{\partial \mathbf{x}}(g(f(\mathbf{x}))) = \frac{\partial g}{\partial f} \frac{\partial f}{\partial \mathbf{x}} \quad (5.48)$$

This is only an intuition, but not mathematically correct since the partial derivative is not a fraction.

Let us have a closer look at the chain rule. The chain rule (5.48) resembles to some degree the rules for matrix multiplication where we said that neighboring dimensions have to match for matrix multiplication to be defined; see Section 2.2.1. If we go from left to right, the chain rule exhibits similar properties: ∂f shows up in the “denominator” of the first factor and in the “numerator” of the second factor. If we multiply the factors together, multiplication is defined, i.e., the dimensions of ∂f match, and ∂f “cancels”, such that $\partial g / \partial \mathbf{x}$ remains.

5.2.2 Chain Rule

Consider a function $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ of two variables x_1, x_2 . Furthermore, $x_1(t)$ and $x_2(t)$ are themselves functions of t . To compute the gradient of f with respect to t , we need to apply the chain rule (5.48) for multivariate functions as

$$\frac{df}{dt} = \begin{bmatrix} \frac{\partial f}{\partial x_1} & \frac{\partial f}{\partial x_2} \end{bmatrix} \begin{bmatrix} \frac{\partial x_1(t)}{\partial t} \\ \frac{\partial x_2(t)}{\partial t} \end{bmatrix} = \frac{\partial f}{\partial x_1} \frac{\partial x_1}{\partial t} + \frac{\partial f}{\partial x_2} \frac{\partial x_2}{\partial t}, \quad (5.49)$$

where d denotes the gradient and ∂ partial derivatives.

Example 5.8

Consider $f(x_1, x_2) = x_1^2 + 2x_2$, where $x_1 = \sin t$ and $x_2 = \cos t$, then

$$\frac{df}{dt} = \frac{\partial f}{\partial x_1} \frac{\partial x_1}{\partial t} + \frac{\partial f}{\partial x_2} \frac{\partial x_2}{\partial t} \quad (5.50a)$$

$$= 2 \sin t \frac{\partial \sin t}{\partial t} + 2 \frac{\partial \cos t}{\partial t} \quad (5.50b)$$

$$= 2 \sin t \cos t - 2 \sin t = 2 \sin t (\cos t - 1) \quad (5.50c)$$

is the corresponding derivative of f with respect to t .

If $f(x_1, x_2)$ is a function of x_1 and x_2 , where $x_1(s, t)$ and $x_2(s, t)$ are themselves functions of two variables s and t , the chain rule yields the partial derivatives

$$\frac{\partial f}{\partial s} = \frac{\partial f}{\partial x_1} \frac{\partial x_1}{\partial s} + \frac{\partial f}{\partial x_2} \frac{\partial x_2}{\partial s}, \quad (5.51)$$

$$\frac{\partial f}{\partial t} = \frac{\partial f}{\partial x_1} \frac{\partial x_1}{\partial t} + \frac{\partial f}{\partial x_2} \frac{\partial x_2}{\partial t}, \quad (5.52)$$

and the gradient is obtained by the matrix multiplication

$$\frac{df}{d(s, t)} = \frac{\partial f}{\partial x} \frac{\partial x}{\partial(s, t)} = \underbrace{\begin{bmatrix} \frac{\partial f}{\partial x_1} & \frac{\partial f}{\partial x_2} \end{bmatrix}}_{\frac{\partial f}{\partial x}} \underbrace{\begin{bmatrix} \frac{\partial x_1}{\partial s} & \frac{\partial x_1}{\partial t} \\ \frac{\partial x_2}{\partial s} & \frac{\partial x_2}{\partial t} \end{bmatrix}}_{\frac{\partial x}{\partial(s, t)}}. \quad (5.53)$$

This compact way of writing the chain rule as a matrix multiplication only makes sense if the gradient is defined as a row vector. Otherwise, we will need to start transposing gradients for the matrix dimensions to match. This may still be straightforward as long as the gradient is a vector or a matrix; however, when the gradient becomes a tensor (we will discuss this in the following), the transpose is no longer a triviality.

The chain rule can be written as a matrix multiplication.

Remark (Verifying the Correctness of a Gradient Implementation). The definition of the partial derivatives as the limit of the corresponding difference quotient (see (5.39)) can be exploited when numerically checking the correctness of gradients in computer programs: When we compute gradients and implement them, we can use finite differences to numerically test our computation and implementation: We choose the value h to be small (e.g., $h = 10^{-4}$) and compare the finite-difference approximation from (5.39) with our (analytic) implementation of the gradient. If the error is small, our gradient implementation is probably correct. “Small” could mean that $\sqrt{\frac{\sum_i (dh_i - df_i)^2}{\sum_i (dh_i + df_i)^2}} < 10^{-6}$, where dh_i is the finite-difference approximation and df_i is the analytic gradient of f with respect to the i th variable x_i . $\diamond$

Gradient checking

5.3 Gradients of Vector-Valued Functions

Thus far, we discussed partial derivatives and gradients of functions $f : \mathbb{R}^n \rightarrow \mathbb{R}$ mapping to the real numbers. In the following, we will generalize the concept of the gradient to vector-valued functions (vector fields) $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$, where $n \geq 1$ and $m > 1$.

For a function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ and a vector $x = [x_1, \dots, x_n]^\top \in \mathbb{R}^n$, the corresponding vector of function values is given as

$$f(x) = \begin{bmatrix} f_1(x) \\ \vdots \\ f_m(x) \end{bmatrix} \in \mathbb{R}^m. \quad (5.54)$$

Writing the vector-valued function in this way allows us to view a vector-valued function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ as a vector of functions $[f_1, \dots, f_m]^\top$, $f_i : \mathbb{R}^n \rightarrow \mathbb{R}$ that map onto $\mathbb{R}$. The differentiation rules for every f_i are exactly the ones we discussed in Section 5.2.

Therefore, the partial derivative of a vector-valued function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ with respect to $x_i \in \mathbb{R}$, $i = 1, \dots, n$, is given as the vector

$$\frac{\partial f}{\partial x_i} = \begin{bmatrix} \frac{\partial f_1}{\partial x_i} \\ \vdots \\ \frac{\partial f_m}{\partial x_i} \end{bmatrix} = \begin{bmatrix} \lim_{h \rightarrow 0} \frac{f_1(x_1, \dots, x_{i-1}, x_i + h, x_{i+1}, \dots, x_n) - f_1(x)}{h} \\ \vdots \\ \lim_{h \rightarrow 0} \frac{f_m(x_1, \dots, x_{i-1}, x_i + h, x_{i+1}, \dots, x_n) - f_m(x)}{h} \end{bmatrix} \in \mathbb{R}^m. \quad (5.55)$$

From (5.40), we know that the gradient of f with respect to a vector is the row vector of the partial derivatives. In (5.55), every partial derivative $\partial f / \partial x_i$ is itself a column vector. Therefore, we obtain the gradient of $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ with respect to $x \in \mathbb{R}^n$ by collecting these partial derivatives:

$$\frac{df(x)}{dx} = \left[\boxed{\frac{\partial f(x)}{\partial x_1}} \dots \boxed{\frac{\partial f(x)}{\partial x_n}} \right] \quad (5.56a)$$

$$= \begin{bmatrix} \boxed{\frac{\partial f_1(x)}{\partial x_1}} & \dots & \boxed{\frac{\partial f_1(x)}{\partial x_n}} \\ \vdots & & \vdots \\ \boxed{\frac{\partial f_m(x)}{\partial x_1}} & \dots & \boxed{\frac{\partial f_m(x)}{\partial x_n}} \end{bmatrix} \in \mathbb{R}^{m \times n}. \quad (5.56b)$$

Definition 5.6 (Jacobian). The collection of all first-order partial derivatives of a vector-valued function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is called the *Jacobian*. The Jacobian J is an $m \times n$ matrix, which we define and arrange as follows:

$$J = \nabla_x f = \frac{df(x)}{dx} = \left[\frac{\partial f(x)}{\partial x_1} \quad \dots \quad \frac{\partial f(x)}{\partial x_n} \right] \quad (5.57)$$

$$= \begin{bmatrix} \frac{\partial f_1(x)}{\partial x_1} & \dots & \frac{\partial f_1(x)}{\partial x_n} \\ \vdots & & \vdots \\ \frac{\partial f_m(x)}{\partial x_1} & \dots & \frac{\partial f_m(x)}{\partial x_n} \end{bmatrix}, \quad (5.58)$$

$$x = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}, \quad J(i, j) = \frac{\partial f_i}{\partial x_j}. \quad (5.59)$$

As a special case of (5.58), a function $f : \mathbb{R}^n \rightarrow \mathbb{R}^1$, which maps a vector $x \in \mathbb{R}^n$ onto a scalar (e.g., $f(x) = \sum_{i=1}^n x_i$), possesses a Jacobian that is a row vector (matrix of dimension $1 \times n$); see (5.40).

Remark. In this book, we use the *numerator layout* of the derivative, i.e., the derivative df/dx of $f \in \mathbb{R}^m$ with respect to $x \in \mathbb{R}^n$ is an $m \times n$ matrix, where the elements of f define the rows and the elements of x define the columns of the corresponding Jacobian; see (5.58). There

Jacobian

The gradient of a function

$f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is a matrix of size $m \times n$.

numerator layout

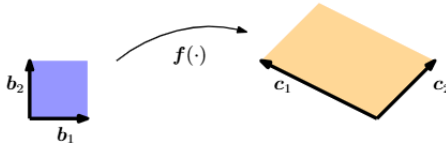


Figure 5.5 The determinant of the Jacobian of f can be used to compute the magnifier between the blue and orange area.

exists also the *denominator layout*, which is the transpose of the numerator layout. In this book, we will use the numerator layout. $\diamond$

We will see how the Jacobian is used in the change-of-variable method for probability distributions in Section 6.7. The amount of scaling due to the transformation of a variable is provided by the determinant.

In Section 4.1, we saw that the determinant can be used to compute the area of a parallelogram. If we are given two vectors $b_1 = [1, 0]^T$, $b_2 = [0, 1]^T$ as the sides of the unit square (blue; see Figure 5.5), the area of this square is

$$\left| \det \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \right| = 1. \quad (5.60)$$

If we take a parallelogram with the sides $c_1 = [-2, 1]^T$, $c_2 = [1, 1]^T$ (orange in Figure 5.5), its area is given as the absolute value of the determinant (see Section 4.1)

$$\left| \det \begin{pmatrix} -2 & 1 \\ 1 & 1 \end{pmatrix} \right| = |-3| = 3, \quad (5.61)$$

i.e., the area of this is exactly three times the area of the unit square. We can find this scaling factor by finding a mapping that transforms the unit square into the other square. In linear algebra terms, we effectively perform a variable transformation from (b_1, b_2) to (c_1, c_2) . In our case, the mapping is linear and the absolute value of the determinant of this mapping gives us exactly the scaling factor we are looking for.

We will describe two approaches to identify this mapping. First, we exploit that the mapping is linear so that we can use the tools from Chapter 2 to identify this mapping. Second, we will find the mapping using partial derivatives using the tools we have been discussing in this chapter.

Approach 1 To get started with the linear algebra approach, we identify both $\{b_1, b_2\}$ and $\{c_1, c_2\}$ as bases of $\mathbb{R}^2$ (see Section 2.6.1 for a recap). What we effectively perform is a change of basis from (b_1, b_2) to (c_1, c_2) , and we are looking for the transformation matrix that implements the basis change. Using results from Section 2.7.2, we identify the desired basis change matrix as

$$J = \begin{bmatrix} -2 & 1 \\ 1 & 1 \end{bmatrix}, \quad (5.62)$$

such that $Jb_1 = c_1$ and $Jb_2 = c_2$. The absolute value of the determi-

denominator layout

nant of J , which yields the scaling factor we are looking for, is given as $|\det(J)| = 3$, i.e., the area of the square spanned by (c_1, c_2) is three times greater than the area spanned by (b_1, b_2) .

Approach 2 The linear algebra approach works for linear transformations; for nonlinear transformations (which become relevant in Section 6.7), we follow a more general approach using partial derivatives.

For this approach, we consider a function $f : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ that performs a variable transformation. In our example, f maps the coordinate representation of any vector $x \in \mathbb{R}^2$ with respect to (b_1, b_2) onto the coordinate representation $y \in \mathbb{R}^2$ with respect to (c_1, c_2) . We want to identify the mapping so that we can compute how an area (or volume) changes when it is being transformed by f . For this, we need to find out how $f(x)$ changes if we modify x a bit. This question is exactly answered by the Jacobian matrix $\frac{df}{dx} \in \mathbb{R}^{2 \times 2}$. Since we can write

$$y_1 = -2x_1 + x_2 \quad (5.63)$$

$$y_2 = x_1 + x_2 \quad (5.64)$$

we obtain the functional relationship between x and y , which allows us to get the partial derivatives

$$\frac{\partial y_1}{\partial x_1} = -2, \quad \frac{\partial y_1}{\partial x_2} = 1, \quad \frac{\partial y_2}{\partial x_1} = 1, \quad \frac{\partial y_2}{\partial x_2} = 1 \quad (5.65)$$

and compose the Jacobian as

$$J = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_1}{\partial x_2} \\ \frac{\partial y_2}{\partial x_1} & \frac{\partial y_2}{\partial x_2} \end{bmatrix} = \begin{bmatrix} -2 & 1 \\ 1 & 1 \end{bmatrix}. \quad (5.66)$$

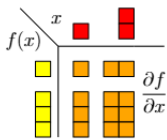
Geometrically, the Jacobian determinant gives the magnification/ scaling factor when we transform an area or volume. Jacobian determinant

The Jacobian represents the coordinate transformation we are looking for. It is exact if the coordinate transformation is linear (as in our case), and (5.66) recovers exactly the basis change matrix in (5.62). If the coordinate transformation is nonlinear, the Jacobian approximates this nonlinear transformation locally with a linear one. The absolute value of the *Jacobian determinant* $|\det(J)|$ is the factor by which areas or volumes are scaled when coordinates are transformed. Our case yields $|\det(J)| = 3$.

The Jacobian determinant and variable transformations will become relevant in Section 6.7 when we transform random variables and probability distributions. These transformations are extremely relevant in machine learning in the context of training deep neural networks using the *reparametrization trick*, also called *infinite perturbation analysis*.

In this chapter, we encountered derivatives of functions. Figure 5.6 summarizes the dimensions of those derivatives. If $f : \mathbb{R} \rightarrow \mathbb{R}$ the gradient is simply a scalar (top-left entry). For $f : \mathbb{R}^D \rightarrow \mathbb{R}$ the gradient is a $1 \times D$ row vector (top-right entry). For $f : \mathbb{R} \rightarrow \mathbb{R}^E$, the gradient is an $E \times 1$ column vector, and for $f : \mathbb{R}^D \rightarrow \mathbb{R}^E$ the gradient is an $E \times D$ matrix.

Figure 5.6
Dimensionality of (partial) derivatives.



Example 5.9 (Gradient of a Vector-Valued Function)

We are given

$$f(x) = Ax, \quad f(x) \in \mathbb{R}^M, \quad A \in \mathbb{R}^{M \times N}, \quad x \in \mathbb{R}^N.$$

To compute the gradient df/dx we first determine the dimension of df/dx : Since $f : \mathbb{R}^N \rightarrow \mathbb{R}^M$, it follows that $df/dx \in \mathbb{R}^{M \times N}$. Second, to compute the gradient we determine the partial derivatives of f with respect to every x_j :

$$f_i(x) = \sum_{j=1}^N A_{ij}x_j \implies \frac{\partial f_i}{\partial x_j} = A_{ij} \quad (5.67)$$

We collect the partial derivatives in the Jacobian and obtain the gradient

$$\frac{df}{dx} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_N} \\ \vdots & & \vdots \\ \frac{\partial f_M}{\partial x_1} & \cdots & \frac{\partial f_M}{\partial x_N} \end{bmatrix} = \begin{bmatrix} A_{11} & \cdots & A_{1N} \\ \vdots & & \vdots \\ A_{M1} & \cdots & A_{MN} \end{bmatrix} = A \in \mathbb{R}^{M \times N}. \quad (5.68)$$

Example 5.10 (Chain Rule)

Consider the function $h : \mathbb{R} \rightarrow \mathbb{R}$, $h(t) = (f \circ g)(t)$ with

$$f : \mathbb{R}^2 \rightarrow \mathbb{R} \quad (5.69)$$

$$g : \mathbb{R} \rightarrow \mathbb{R}^2 \quad (5.70)$$

$$f(x) = \exp(x_1 x_2^2), \quad (5.71)$$

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = g(t) = \begin{bmatrix} t \cos t \\ t \sin t \end{bmatrix} \quad (5.72)$$

and compute the gradient of h with respect to t . Since $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ and $g : \mathbb{R} \rightarrow \mathbb{R}^2$ we note that

$$\frac{\partial f}{\partial x} \in \mathbb{R}^{1 \times 2}, \quad \frac{\partial g}{\partial t} \in \mathbb{R}^{2 \times 1}. \quad (5.73)$$

The desired gradient is computed by applying the chain rule:

$$\frac{dh}{dt} = \frac{\partial f}{\partial x} \frac{\partial x}{\partial t} = \begin{bmatrix} \frac{\partial f}{\partial x_1} & \frac{\partial f}{\partial x_2} \end{bmatrix} \begin{bmatrix} \frac{\partial x_1}{\partial t} \\ \frac{\partial x_2}{\partial t} \end{bmatrix} \quad (5.74a)$$

$$= [\exp(x_1 x_2^2) x_2^2 \quad 2 \exp(x_1 x_2^2) x_1 x_2] \begin{bmatrix} \cos t - t \sin t \\ \sin t + t \cos t \end{bmatrix} \quad (5.74b)$$

$$= \exp(x_1 x_2^2) (x_2^2 (\cos t - t \sin t) + 2x_1 x_2 (\sin t + t \cos t)), \quad (5.74c)$$

where $x_1 = t \cos t$ and $x_2 = t \sin t$; see (5.72).

We will discuss this model in much more detail in Chapter 9 in the context of linear regression, where we need derivatives of the least-squares loss L with respect to the parameters θ .

least-squares loss

```
dLdtheta =
np.einsum(
'n,nd',
dLde,dedtheta)
```

Example 5.11 (Gradient of a Least-Squares Loss in a Linear Model)

Let us consider the linear model

$$\mathbf{y} = \Phi \boldsymbol{\theta}, \quad (5.75)$$

where $\boldsymbol{\theta} \in \mathbb{R}^D$ is a parameter vector, $\Phi \in \mathbb{R}^{N \times D}$ are input features and $\mathbf{y} \in \mathbb{R}^N$ are the corresponding observations. We define the functions

$$L(\mathbf{e}) := \|\mathbf{e}\|^2, \quad (5.76)$$

$$\mathbf{e}(\boldsymbol{\theta}) := \mathbf{y} - \Phi \boldsymbol{\theta}. \quad (5.77)$$

We seek $\frac{\partial L}{\partial \boldsymbol{\theta}}$, and we will use the chain rule for this purpose. L is called a *least-squares loss* function.

Before we start our calculation, we determine the dimensionality of the gradient as

$$\frac{\partial L}{\partial \boldsymbol{\theta}} \in \mathbb{R}^{1 \times D}. \quad (5.78)$$

The chain rule allows us to compute the gradient as

$$\frac{\partial L}{\partial \boldsymbol{\theta}} = \frac{\partial L}{\partial \mathbf{e}} \frac{\partial \mathbf{e}}{\partial \boldsymbol{\theta}}, \quad (5.79)$$

where the d th element is given by

$$\frac{\partial L}{\partial \boldsymbol{\theta}}[1, d] = \sum_{n=1}^N \frac{\partial L}{\partial \mathbf{e}}[n] \frac{\partial \mathbf{e}}{\partial \boldsymbol{\theta}}[n, d]. \quad (5.80)$$

We know that $\|\mathbf{e}\|^2 = \mathbf{e}^\top \mathbf{e}$ (see Section 3.2) and determine

$$\frac{\partial L}{\partial \mathbf{e}} = 2\mathbf{e}^\top \in \mathbb{R}^{1 \times N}. \quad (5.81)$$

Furthermore, we obtain

$$\frac{\partial \mathbf{e}}{\partial \boldsymbol{\theta}} = -\Phi \in \mathbb{R}^{N \times D}, \quad (5.82)$$

such that our desired derivative is

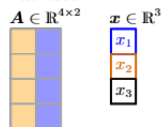
$$\frac{\partial L}{\partial \boldsymbol{\theta}} = -2\mathbf{e}^\top \Phi \stackrel{(5.77)}{=} -\underbrace{2(\mathbf{y}^\top - \boldsymbol{\theta}^\top \Phi^\top)}_{1 \times N} \underbrace{\Phi}_{N \times D} \in \mathbb{R}^{1 \times D}. \quad (5.83)$$

Remark. We would have obtained the same result without using the chain rule by immediately looking at the function

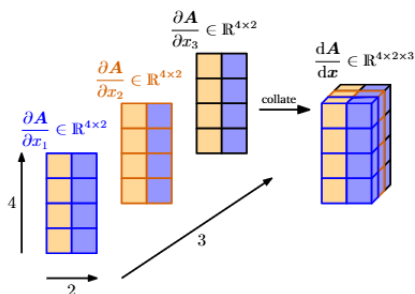
$$L_2(\boldsymbol{\theta}) := \|\mathbf{y} - \Phi \boldsymbol{\theta}\|^2 = (\mathbf{y} - \Phi \boldsymbol{\theta})^\top (\mathbf{y} - \Phi \boldsymbol{\theta}). \quad (5.84)$$

This approach is still practical for simple functions like L_2 but becomes impractical for deep function compositions. $\diamond$

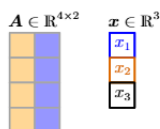
5.4 Gradients of Matrices

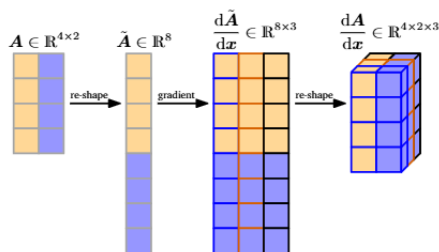
$$A \in \mathbb{R}^{4 \times 2} \quad x \in \mathbb{R}^3$$


Partial derivatives:



(a) Approach 1: We compute the partial derivative $\frac{\partial A}{\partial x_1}$, $\frac{\partial A}{\partial x_2}$, $\frac{\partial A}{\partial x_3}$, each of which is a 4×2 matrix, and collate them in a $4 \times 2 \times 3$ tensor.

$$A \in \mathbb{R}^{4 \times 2} \quad x \in \mathbb{R}^3$$




(b) Approach 2: We re-shape (flatten) $A \in \mathbb{R}^{4 \times 2}$ into a vector $\tilde{A} \in \mathbb{R}^8$. Then, we compute the gradient $\frac{d\tilde{A}}{dx} \in \mathbb{R}^{8 \times 3}$. We obtain the gradient tensor by re-shaping this gradient as illustrated above.

5.4 Gradients of Matrices

We will encounter situations where we need to take gradients of matrices with respect to vectors (or other matrices), which results in a multidimensional tensor. We can think of this tensor as a multidimensional array that

155

Figure 5.7
Visualization of gradient computation of a matrix with respect to a vector. We are interested in computing the gradient of $A \in \mathbb{R}^{4 \times 2}$ with respect to a vector $x \in \mathbb{R}^3$. We know that gradient $\frac{dA}{dx} \in \mathbb{R}^{4 \times 2 \times 3}$. We follow two equivalent approaches to arrive there: (a) collating partial derivatives into a Jacobian tensor; (b) flattening of the matrix into a vector, computing the Jacobian matrix, re-shaping into a Jacobian tensor.

We can think of a tensor as a multidimensional array.

collects partial derivatives. For example, if we compute the gradient of an $m \times n$ matrix A with respect to a $p \times q$ matrix B , the resulting Jacobian would be $(m \times n) \times (p \times q)$, i.e., a four-dimensional tensor J , whose entries are given as $J_{ijkl} = \partial A_{ij} / \partial B_{kl}$.

Since matrices represent linear mappings, we can exploit the fact that there is a vector-space isomorphism (linear, invertible mapping) between the space $\mathbb{R}^{m \times n}$ of $m \times n$ matrices and the space $\mathbb{R}^{mn}$ of mn vectors. Therefore, we can re-shape our matrices into vectors of lengths mn and pq , respectively. The gradient using these mn vectors results in a Jacobian of size $mn \times pq$. Figure 5.7 visualizes both approaches. In practical applications, it is often desirable to re-shape the matrix into a vector and continue working with this Jacobian matrix: The chain rule (5.48) boils down to simple matrix multiplication, whereas in the case of a Jacobian tensor, we will need to pay more attention to what dimensions we need to sum out.

Matrices can be transformed into vectors by stacking the columns of the matrix ("flattening").

Example 5.12 (Gradient of Vectors with Respect to Matrices)

Let us consider the following example, where

$$f = Ax, \quad f \in \mathbb{R}^M, \quad A \in \mathbb{R}^{M \times N}, \quad x \in \mathbb{R}^N \quad (5.85)$$

and where we seek the gradient df/dA . Let us start again by determining the dimension of the gradient as

$$\frac{df}{dA} \in \mathbb{R}^{M \times (M \times N)}. \quad (5.86)$$

By definition, the gradient is the collection of the partial derivatives:

$$\frac{df}{dA} = \begin{bmatrix} \frac{\partial f_1}{\partial A} \\ \vdots \\ \frac{\partial f_M}{\partial A} \end{bmatrix}, \quad \frac{\partial f_i}{\partial A} \in \mathbb{R}^{1 \times (M \times N)}. \quad (5.87)$$

To compute the partial derivatives, it will be helpful to explicitly write out the matrix vector multiplication:

$$f_i = \sum_{j=1}^N A_{ij} x_j, \quad i = 1, \dots, M, \quad (5.88)$$

and the partial derivatives are then given as

$$\frac{\partial f_i}{\partial A_{iq}} = x_q. \quad (5.89)$$

This allows us to compute the partial derivatives of f_i with respect to a row of A , which is given as

$$\frac{\partial f_i}{\partial A_{i,:}} = x^\top \in \mathbb{R}^{1 \times 1 \times N}, \quad (5.90)$$

$$\frac{\partial f_i}{\partial A_{k \neq i,:}} = \mathbf{0}^\top \in \mathbb{R}^{1 \times 1 \times N} \quad (5.91)$$

where we have to pay attention to the correct dimensionality. Since f_i maps onto $\mathbb{R}$ and each row of A is of size $1 \times N$, we obtain a $1 \times 1 \times N$ -sized tensor as the partial derivative of f_i with respect to a row of A .

We stack the partial derivatives (5.91) and get the desired gradient in (5.87) via

$$\frac{\partial f_i}{\partial A} = \begin{bmatrix} \mathbf{0}^\top \\ \vdots \\ \mathbf{0}^\top \\ x^\top \\ \mathbf{0}^\top \\ \vdots \\ \mathbf{0}^\top \end{bmatrix} \in \mathbb{R}^{1 \times (M \times N)}. \quad (5.92)$$

Example 5.13 (Gradient of Matrices with Respect to Matrices)

Consider a matrix $R \in \mathbb{R}^{M \times N}$ and $f : \mathbb{R}^{M \times N} \rightarrow \mathbb{R}^{N \times N}$ with

$$f(R) = R^\top R =: K \in \mathbb{R}^{N \times N}, \quad (5.93)$$

where we seek the gradient dK/dR .

To solve this hard problem, let us first write down what we already know: The gradient has the dimensions

$$\frac{dK}{dR} \in \mathbb{R}^{(N \times N) \times (M \times N)}, \quad (5.94)$$

which is a tensor. Moreover,

$$\frac{dK_{pq}}{dR} \in \mathbb{R}^{1 \times M \times N} \quad (5.95)$$

for $p, q = 1, \dots, N$, where K_{pq} is the (p, q) th entry of $K = f(R)$. Denoting the i th column of R by r_i , every entry of K is given by the dot product of two columns of R , i.e.,

$$K_{pq} = r_p^\top r_q = \sum_{m=1}^M R_{mp} R_{mq}. \quad (5.96)$$

When we now compute the partial derivative $\frac{\partial K_{pq}}{\partial R_{ij}}$ we obtain

$$\frac{\partial K_{pq}}{\partial R_{ij}} = \sum_{m=1}^M \frac{\partial}{\partial R_{ij}} R_{mp} R_{mq} = \partial_{pqij}, \quad (5.97)$$

$$\partial_{pqij} = \begin{cases} R_{iq} & \text{if } j = p, p \neq q \\ R_{ip} & \text{if } j = q, p \neq q \\ 2R_{iq} & \text{if } j = p, p = q \\ 0 & \text{otherwise} \end{cases} \quad (5.98)$$

From (5.94), we know that the desired gradient has the dimension $(N \times N) \times (M \times N)$, and every single entry of this tensor is given by ∂_{pqij} in (5.98), where $p, q, j = 1, \dots, N$ and $i = 1, \dots, M$.

5.5 Useful Identities for Computing Gradients

In the following, we list some useful gradients that are frequently required in a machine learning context (Petersen and Pedersen, 2012). Here, we use $\text{tr}(\cdot)$ as the trace (see Definition 4.4), $\det(\cdot)$ as the determinant (see Section 4.1) and $f(X)^{-1}$ as the inverse of $f(X)$, assuming it exists.

$$\frac{\partial}{\partial X} f(X)^\top = \left(\frac{\partial f(X)}{\partial X} \right)^\top \quad (5.99)$$

$$\frac{\partial}{\partial X} \text{tr}(f(X)) = \text{tr} \left(\frac{\partial f(X)}{\partial X} \right) \quad (5.100)$$

$$\frac{\partial}{\partial X} \det(f(X)) = \det(f(X)) \text{tr} \left(f(X)^{-1} \frac{\partial f(X)}{\partial X} \right) \quad (5.101)$$

$$\frac{\partial}{\partial X} f(X)^{-1} = -f(X)^{-1} \frac{\partial f(X)}{\partial X} f(X)^{-1} \quad (5.102)$$

$$\frac{\partial a^\top X^{-1} b}{\partial X} = -(X^{-1})^\top a b^\top (X^{-1})^\top \quad (5.103)$$

$$\frac{\partial x^\top a}{\partial x} = a^\top \quad (5.104)$$

$$\frac{\partial a^\top x}{\partial x} = a^\top \quad (5.105)$$

$$\frac{\partial a^\top X b}{\partial X} = a b^\top \quad (5.106)$$

$$\frac{\partial x^\top B x}{\partial x} = x^\top (B + B^\top) \quad (5.107)$$

$$\frac{\partial}{\partial s} (x - As)^\top W (x - As) = -2(x - As)^\top W A \quad \text{for symmetric } W \quad (5.108)$$

Remark. In this book, we only cover traces and transposes of matrices. However, we have seen that derivatives can be higher-dimensional tensors, in which case the usual trace and transpose are not defined. In these cases, the trace of a $D \times D \times E \times F$ tensor would be an $E \times F$ -dimensional matrix. This is a special case of a tensor contraction. Similarly, when we

“transpose” a tensor, we mean swapping the first two dimensions. Specifically, in (5.99) through (5.102), we require tensor-related computations when we work with multivariate functions $f(\cdot)$ and compute derivatives with respect to matrices (and choose not to vectorize them as discussed in Section 5.4). $\diamond$

5.6 Backpropagation and Automatic Differentiation

In many machine learning applications, we find good model parameters by performing gradient descent (Section 7.1), which relies on the fact that we can compute the gradient of a learning objective with respect to the parameters of the model. For a given objective function, we can obtain the gradient with respect to the model parameters using calculus and applying the chain rule; see Section 5.2.2. We already had a taste in Section 5.3 when we looked at the gradient of a squared loss with respect to the parameters of a linear regression model.

Consider the function

$$f(x) = \sqrt{x^2 + \exp(x^2)} + \cos(x^2 + \exp(x^2)). \quad (5.109)$$

By application of the chain rule, and noting that differentiation is linear, we compute the gradient

$$\begin{aligned} \frac{df}{dx} &= \frac{2x + 2x \exp(x^2)}{2\sqrt{x^2 + \exp(x^2)}} - \sin(x^2 + \exp(x^2)) (2x + 2x \exp(x^2)) \\ &= 2x \left(\frac{1}{2\sqrt{x^2 + \exp(x^2)}} - \sin(x^2 + \exp(x^2)) \right) (1 + \exp(x^2)). \end{aligned} \quad (5.110)$$

Writing out the gradient in this explicit way is often impractical since it often results in a very lengthy expression for a derivative. In practice, it means that, if we are not careful, the implementation of the gradient could be significantly more expensive than computing the function, which imposes unnecessary overhead. For training deep neural network models, the *backpropagation* algorithm (Kelley, 1960; Bryson, 1961; Dreyfus, 1962; Rumelhart et al., 1986) is an efficient way to compute the gradient of an error function with respect to the parameters of the model.

A good discussion about backpropagation and the chain rule is available at a blog by Tim Vieira at <https://tinyurl.com/yycfm2yrw>.

backpropagation

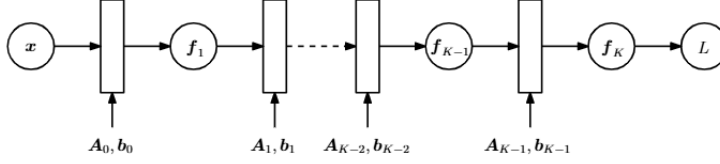
5.6.1 Gradients in a Deep Network

An area where the chain rule is used to an extreme is deep learning, where the function value y is computed as a many-level function composition

$$y = (f_K \circ f_{K-1} \circ \cdots \circ f_1)(x) = f_K(f_{K-1}(\cdots(f_1(x))\cdots)), \quad (5.111)$$

where x are the inputs (e.g., images), y are the observations (e.g., class labels), and every function f_i , $i = 1, \dots, K$, possesses its own parameters.

Figure 5.8 Forward pass in a multi-layer neural network to compute the loss L as a function of the inputs x and the parameters A_i, b_i .



We discuss the case, where the activation functions are identical in each layer to unclutter notation.

In neural networks with multiple layers, we have functions $f_i(x_{i-1}) = \sigma(A_{i-1}x_{i-1} + b_{i-1})$ in the i th layer. Here x_{i-1} is the output of layer $i-1$ and σ an activation function, such as the logistic sigmoid $\frac{1}{1+e^{-x}}$, tanh or a rectified linear unit (ReLU). In order to train these models, we require the gradient of a loss function L with respect to all model parameters A_j, b_j for $j = 1, \dots, K$. This also requires us to compute the gradient of L with respect to the inputs of each layer. For example, if we have inputs x and observations y and a network structure defined by

$$f_0 := x \quad (5.112)$$

$$f_i := \sigma_i(A_{i-1}f_{i-1} + b_{i-1}), \quad i = 1, \dots, K, \quad (5.113)$$

see also Figure 5.8 for a visualization, we may be interested in finding A_j, b_j for $j = 0, \dots, K-1$, such that the squared loss

$$L(\theta) = \|y - f_K(\theta, x)\|^2 \quad (5.114)$$

is minimized, where $\theta = \{A_0, b_0, \dots, A_{K-1}, b_{K-1}\}$.

To obtain the gradients with respect to the parameter set θ , we require the partial derivatives of L with respect to the parameters $\theta_j = \{A_j, b_j\}$ of each layer $j = 0, \dots, K-1$. The chain rule allows us to determine the partial derivatives as

$$\frac{\partial L}{\partial \theta_{K-1}} = \frac{\partial L}{\partial f_K} \frac{\partial f_K}{\partial \theta_{K-1}} \quad (5.115)$$

$$\frac{\partial L}{\partial \theta_{K-2}} = \frac{\partial L}{\partial f_K} \boxed{\frac{\partial f_K}{\partial f_{K-1}}} \frac{\partial f_{K-1}}{\partial \theta_{K-2}} \quad (5.116)$$

$$\frac{\partial L}{\partial \theta_{K-3}} = \frac{\partial L}{\partial f_K} \frac{\partial f_K}{\partial f_{K-1}} \boxed{\frac{\partial f_{K-1}}{\partial f_{K-2}}} \frac{\partial f_{K-2}}{\partial \theta_{K-3}} \quad (5.117)$$

$$\frac{\partial L}{\partial \theta_i} = \frac{\partial L}{\partial f_K} \frac{\partial f_K}{\partial f_{K-1}} \dots \boxed{\frac{\partial f_{i+2}}{\partial f_{i+1}}} \frac{\partial f_{i+1}}{\partial \theta_i} \quad (5.118)$$

The **orange** terms are partial derivatives of the output of a layer with respect to its inputs, whereas the **blue** terms are partial derivatives of the output of a layer with respect to its parameters. Assuming, we have already computed the partial derivatives $\partial L / \partial \theta_{i+1}$, then most of the computation can be reused to compute $\partial L / \partial \theta_i$. The additional terms that we

A more in-depth discussion about gradients of neural networks can be found in Justin Domke's lecture notes <https://tinyurl.com/yalcxgtv>.

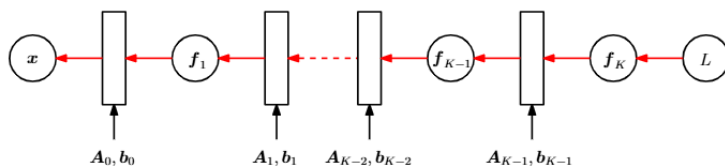


Figure 5.9
Backward pass in a multi-layer neural network to compute the gradients of the loss function.

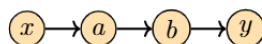


Figure 5.10 Simple graph illustrating the flow of data from x to y via some intermediate variables a, b .

need to compute are indicated by the boxes. Figure 5.9 visualizes that the gradients are passed backward through the network.

5.6.2 Automatic Differentiation

It turns out that backpropagation is a special case of a general technique in numerical analysis called *automatic differentiation*. We can think of automatic differentiation as a set of techniques to numerically (in contrast to symbolically) evaluate the exact (up to machine precision) gradient of a function by working with intermediate variables and applying the chain rule. Automatic differentiation applies a series of elementary arithmetic operations, e.g., addition and multiplication and elementary functions, e.g., $\sin$, $\cos$, $\exp$, $\log$. By applying the chain rule to these operations, the gradient of quite complicated functions can be computed automatically. Automatic differentiation applies to general computer programs and has forward and reverse modes. Baydin et al. (2018) give a great overview of automatic differentiation in machine learning.

Figure 5.10 shows a simple graph representing the data flow from inputs x to outputs y via some intermediate variables a, b . If we were to compute the derivative dy/dx , we would apply the chain rule and obtain

$$\frac{dy}{dx} = \frac{dy}{db} \frac{db}{da} \frac{da}{dx}. \quad (5.119)$$

Intuitively, the forward and reverse mode differ in the order of multiplication. Due to the associativity of matrix multiplication, we can choose between

$$\frac{dy}{dx} = \left(\frac{dy}{db} \frac{db}{da} \right) \frac{da}{dx}, \quad (5.120)$$

$$\frac{dy}{dx} = \frac{dy}{db} \left(\frac{db}{da} \frac{da}{dx} \right). \quad (5.121)$$

Equation (5.120) would be the *reverse mode* because gradients are propagated backward through the graph, i.e., reverse to the data flow. Equation (5.121) would be the *forward mode*, where the gradients flow with the data from left to right through the graph.

automatic differentiation

Automatic differentiation is different from symbolic differentiation and numerical approximations of the gradient, e.g., by using finite differences.

In the general case, we work with Jacobians, which can be vectors, matrices, or tensors.

reverse mode

forward mode

In the following, we will focus on reverse mode automatic differentiation, which is backpropagation. In the context of neural networks, where the input dimensionality is often much higher than the dimensionality of the labels, the reverse mode is computationally significantly cheaper than the forward mode. Let us start with an instructive example.

Example 5.14

Consider the function

$$f(x) = \sqrt{x^2 + \exp(x^2)} + \cos(x^2 + \exp(x^2)) \quad (5.122)$$

from (5.109). If we were to implement a function f on a computer, we would be able to save some computation by using *intermediate variables*:

$$a = x^2, \quad (5.123)$$

$$b = \exp(a), \quad (5.124)$$

$$c = a + b, \quad (5.125)$$

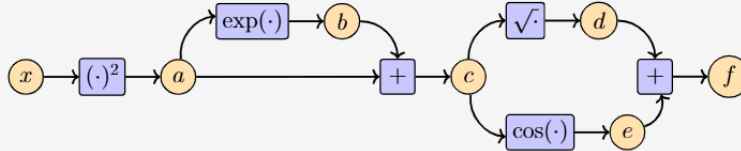
$$d = \sqrt{c}, \quad (5.126)$$

$$e = \cos(c), \quad (5.127)$$

$$f = d + e. \quad (5.128)$$

intermediate
variables

Figure 5.11
Computation graph
with inputs x ,
function values f ,
and intermediate
variables a, b, c, d, e .



This is the same kind of thinking process that occurs when applying the chain rule. Note that the preceding set of equations requires fewer operations than a direct implementation of the function $f(x)$ as defined in (5.109). The corresponding computation graph in Figure 5.11 shows the flow of data and computations required to obtain the function value f .

The set of equations that include intermediate variables can be thought of as a computation graph, a representation that is widely used in implementations of neural network software libraries. We can directly compute the derivatives of the intermediate variables with respect to their corresponding inputs by recalling the definition of the derivative of elementary functions. We obtain the following:

$$\frac{\partial a}{\partial x} = 2x \quad (5.129)$$

$$\frac{\partial b}{\partial a} = \exp(a) \quad (5.130)$$

$$\frac{\partial c}{\partial a} = 1 = \frac{\partial c}{\partial b} \quad (5.131)$$

$$\frac{\partial d}{\partial c} = \frac{1}{2\sqrt{c}} \quad (5.132)$$

$$\frac{\partial e}{\partial c} = -\sin(c) \quad (5.133)$$

$$\frac{\partial f}{\partial d} = 1 = \frac{\partial f}{\partial e}. \quad (5.134)$$

By looking at the computation graph in Figure 5.11, we can compute $\partial f/\partial x$ by working backward from the output and obtain

$$\frac{\partial f}{\partial c} = \frac{\partial f}{\partial d} \frac{\partial d}{\partial c} + \frac{\partial f}{\partial e} \frac{\partial e}{\partial c} \quad (5.135)$$

$$\frac{\partial f}{\partial b} = \frac{\partial f}{\partial c} \frac{\partial c}{\partial b} \quad (5.136)$$

$$\frac{\partial f}{\partial a} = \frac{\partial f}{\partial b} \frac{\partial b}{\partial a} + \frac{\partial f}{\partial c} \frac{\partial c}{\partial a} \quad (5.137)$$

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial a} \frac{\partial a}{\partial x}. \quad (5.138)$$

Note that we implicitly applied the chain rule to obtain $\partial f/\partial x$. By substituting the results of the derivatives of the elementary functions, we get

$$\frac{\partial f}{\partial c} = 1 \cdot \frac{1}{2\sqrt{c}} + 1 \cdot (-\sin(c)) \quad (5.139)$$

$$\frac{\partial f}{\partial b} = \frac{\partial f}{\partial c} \cdot 1 \quad (5.140)$$

$$\frac{\partial f}{\partial a} = \frac{\partial f}{\partial b} \exp(a) + \frac{\partial f}{\partial c} \cdot 1 \quad (5.141)$$

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial a} \cdot 2x. \quad (5.142)$$

By thinking of each of the derivatives above as a variable, we observe that the computation required for calculating the derivative is of similar complexity as the computation of the function itself. This is quite counter-intuitive since the mathematical expression for the derivative $\frac{\partial f}{\partial x}$ (5.110) is significantly more complicated than the mathematical expression of the function $f(x)$ in (5.109).

Automatic differentiation is a formalization of Example 5.14. Let $x_1, \dots, x_d$ be the input variables to the function, $x_{d+1}, \dots, x_{D-1}$ be the intermediate variables, and x_D the output variable. Then the computation graph can be expressed as follows:

$$\text{For } i = d+1, \dots, D: \quad x_i = g_i(x_{\text{Pa}(x_i)}), \quad (5.143)$$

where the $g_i(\cdot)$ are elementary functions and $x_{\text{Pa}(x_i)}$ are the parent nodes of the variable x_i in the graph. Given a function defined in this way, we can use the chain rule to compute the derivative of the function in a step-by-step fashion. Recall that by definition $f = x_D$ and hence

$$\frac{\partial f}{\partial x_D} = 1. \quad (5.144)$$

For other variables x_i , we apply the chain rule

$$\frac{\partial f}{\partial x_i} = \sum_{x_j: x_i \in \text{Pa}(x_j)} \frac{\partial f}{\partial x_j} \frac{\partial x_j}{\partial x_i} = \sum_{x_j: x_i \in \text{Pa}(x_j)} \frac{\partial f}{\partial x_j} \frac{\partial g_j}{\partial x_i}, \quad (5.145)$$

where $\text{Pa}(x_j)$ is the set of parent nodes of x_j in the computation graph. Equation (5.143) is the forward propagation of a function, whereas (5.145) is the backpropagation of the gradient through the computation graph. For neural network training, we backpropagate the error of the prediction with respect to the label.

Auto-differentiation in reverse mode requires a parse tree.

The automatic differentiation approach above works whenever we have a function that can be expressed as a computation graph, where the elementary functions are differentiable. In fact, the function may not even be a mathematical function but a computer program. However, not all computer programs can be automatically differentiated, e.g., if we cannot find differential elementary functions. Programming structures, such as `for` loops and `if` statements, require more care as well.

5.7 Higher-Order Derivatives

So far, we have discussed gradients, i.e., first-order derivatives. Sometimes, we are interested in derivatives of higher order, e.g., when we want to use Newton's Method for optimization, which requires second-order derivatives (Nocedal and Wright, 2006). In Section 5.1.1, we discussed the Taylor series to approximate functions using polynomials. In the multivariate case, we can do exactly the same. In the following, we will do exactly this. But let us start with some notation.

Consider a function $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ of two variables x, y . We use the following notation for higher-order partial derivatives (and for gradients):

- $\frac{\partial^2 f}{\partial x^2}$ is the second partial derivative of f with respect to x .
- $\frac{\partial^n f}{\partial x^n}$ is the n th partial derivative of f with respect to x .
- $\frac{\partial^2 f}{\partial y \partial x} = \frac{\partial}{\partial y} \left(\frac{\partial f}{\partial x} \right)$ is the partial derivative obtained by first partial differentiating with respect to x and then with respect to y .
- $\frac{\partial^2 f}{\partial x \partial y}$ is the partial derivative obtained by first partial differentiating by y and then x .

Hessian

The *Hessian* is the collection of all second-order partial derivatives.

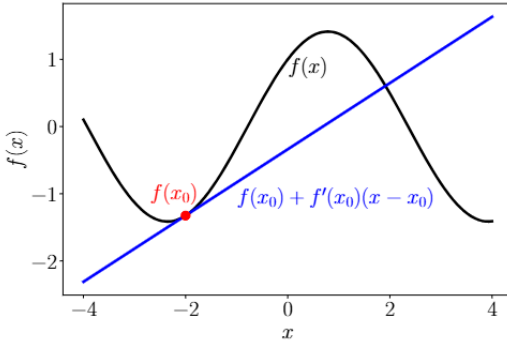


Figure 5.12 Linear approximation of a function. The original function f is linearized at $x_0 = -2$ using a first-order Taylor series expansion.

If $f(x, y)$ is a twice (continuously) differentiable function, then

$$\frac{\partial^2 f}{\partial x \partial y} = \frac{\partial^2 f}{\partial y \partial x}, \quad (5.146)$$

i.e., the order of differentiation does not matter, and the corresponding *Hessian matrix*

Hessian matrix

$$H = \begin{bmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial x \partial y} & \frac{\partial^2 f}{\partial y^2} \end{bmatrix} \quad (5.147)$$

is symmetric. The Hessian is denoted as $\nabla_{x,y}^2 f(x, y)$. Generally, for $x \in \mathbb{R}^n$ and $f : \mathbb{R}^n \rightarrow \mathbb{R}$, the Hessian is an $n \times n$ matrix. The Hessian measures the curvature of the function locally around (x, y) .

Remark (Hessian of a Vector Field). If $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is a vector field, the Hessian is an $(m \times n \times n)$ -tensor. $\diamond$

5.8 Linearization and Multivariate Taylor Series

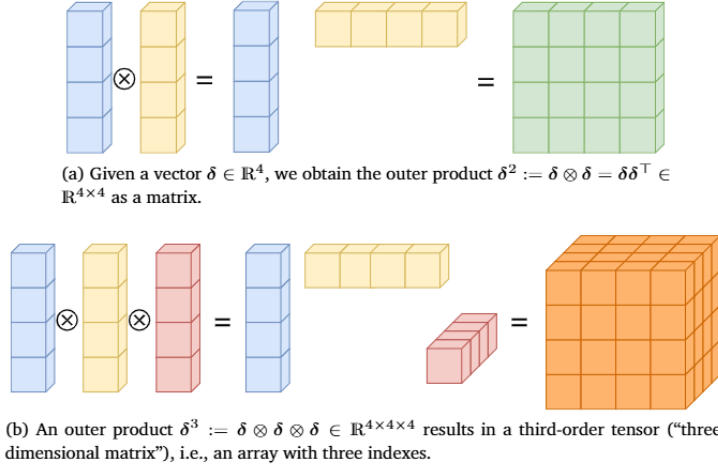
The gradient ∇f of a function f is often used for a locally linear approximation of f around x_0 :

$$f(x) \approx f(x_0) + (\nabla_x f)(x_0)(x - x_0). \quad (5.148)$$

Here $(\nabla_x f)(x_0)$ is the gradient of f with respect to x , evaluated at x_0 . Figure 5.12 illustrates the linear approximation of a function f at an input x_0 . The original function is approximated by a straight line. This approximation is locally accurate, but the farther we move away from x_0 the worse the approximation gets. Equation (5.148) is a special case of a multivariate Taylor series expansion of f at x_0 , where we consider only the first two terms. We discuss the more general case in the following, which will allow for better approximations.

Figure 5.13

Visualizing outer products. Outer products of vectors increase the dimensionality of the array by 1 per term. (a) The outer product of two vectors results in a matrix; (b) the outer product of three vectors yields a third-order tensor.



Definition 5.7 (Multivariate Taylor Series). We consider a function

$$f : \mathbb{R}^D \rightarrow \mathbb{R} \quad (5.149)$$

$$x \mapsto f(x), \quad x \in \mathbb{R}^D, \quad (5.150)$$

that is smooth at x_0 . When we define the difference vector $\delta := x - x_0$, the *multivariate Taylor series* of f at (x_0) is defined as

$$f(x) = \sum_{k=0}^{\infty} \frac{D_x^k f(x_0)}{k!} \delta^k, \quad (5.151)$$

where $D_x^k f(x_0)$ is the k -th (total) derivative of f with respect to x , evaluated at x_0 .

Definition 5.8 (Taylor Polynomial). The *Taylor polynomial* of degree n of f at x_0 contains the first $n + 1$ components of the series in (5.151) and is defined as

$$T_n(x) = \sum_{k=0}^n \frac{D_x^k f(x_0)}{k!} \delta^k. \quad (5.152)$$

In (5.151) and (5.152), we used the slightly sloppy notation of δ^k , which is not defined for vectors $x \in \mathbb{R}^D$, $D > 1$, and $k > 1$. Note that both $D_x^k f$ and δ^k are k -th order tensors, i.e., k -dimensional arrays. The

k th-order tensor $\delta^k \in \mathbb{R}^{\overbrace{D \times D \times \dots \times D}^{k \text{ times}}}$ is obtained as a k -fold outer product, denoted by $\otimes$, of the vector $\delta \in \mathbb{R}^D$. For example,

$$\delta^2 := \delta \otimes \delta = \delta \delta^\top, \quad \delta^2[i, j] = \delta[i] \delta[j] \quad (5.153)$$

$$\delta^3 := \delta \otimes \delta \otimes \delta, \quad \delta^3[i, j, k] = \delta[i]\delta[j]\delta[k]. \quad (5.154)$$

Figure 5.13 visualizes two such outer products. In general, we obtain the terms

$$D_{\mathbf{x}}^k f(\mathbf{x}_0) \delta^k = \sum_{i_1=1}^D \cdots \sum_{i_k=1}^D D_{\mathbf{x}}^k f(\mathbf{x}_0)[i_1, \dots, i_k] \delta[i_1] \cdots \delta[i_k] \quad (5.155)$$

in the Taylor series, where $D_{\mathbf{x}}^k f(\mathbf{x}_0) \delta^k$ contains k -th order polynomials.

Now that we defined the Taylor series for vector fields, let us explicitly write down the first terms $D_{\mathbf{x}}^k f(\mathbf{x}_0) \delta^k$ of the Taylor series expansion for $k = 0, \dots, 3$ and $\delta := \mathbf{x} - \mathbf{x}_0$:

$$k = 0 : D_{\mathbf{x}}^0 f(\mathbf{x}_0) \delta^0 = f(\mathbf{x}_0) \in \mathbb{R} \quad (5.156)$$

$$k = 1 : D_{\mathbf{x}}^1 f(\mathbf{x}_0) \delta^1 = \underbrace{\nabla_{\mathbf{x}} f(\mathbf{x}_0)}_{1 \times D} \underbrace{\delta}_{D \times 1} = \sum_{i=1}^D \nabla_{\mathbf{x}} f(\mathbf{x}_0)[i] \delta[i] \in \mathbb{R} \quad (5.157)$$

$$k = 2 : D_{\mathbf{x}}^2 f(\mathbf{x}_0) \delta^2 = \text{tr} \left(\underbrace{H(\mathbf{x}_0)}_{D \times D} \underbrace{\delta}_{D \times 1} \underbrace{\delta^{\top}}_{1 \times D} \right) = \delta^{\top} H(\mathbf{x}_0) \delta \quad (5.158)$$

$$= \sum_{i=1}^D \sum_{j=1}^D H[i, j] \delta[i] \delta[j] \in \mathbb{R} \quad (5.159)$$

$$k = 3 : D_{\mathbf{x}}^3 f(\mathbf{x}_0) \delta^3 = \sum_{i=1}^D \sum_{j=1}^D \sum_{k=1}^D D_{\mathbf{x}}^3 f(\mathbf{x}_0)[i, j, k] \delta[i] \delta[j] \delta[k] \in \mathbb{R} \quad (5.160)$$

Here, $H(\mathbf{x}_0)$ is the Hessian of f evaluated at $\mathbf{x}_0$.

Example 5.15 (Taylor Series Expansion of a Function with Two Variables)

Consider the function

$$f(x, y) = x^2 + 2xy + y^3. \quad (5.161)$$

We want to compute the Taylor series expansion of f at $(x_0, y_0) = (1, 2)$. Before we start, let us discuss what to expect: The function in (5.161) is a polynomial of degree 3. We are looking for a Taylor series expansion, which itself is a linear combination of polynomials. Therefore, we do not expect the Taylor series expansion to contain terms of fourth or higher order to express a third-order polynomial. This means that it should be sufficient to determine the first four terms of (5.151) for an exact alternative representation of (5.161).

To determine the Taylor series expansion, we start with the constant term and the first-order derivatives, which are given by

$$f(1, 2) = 13 \quad (5.162)$$

```
np.einsum('i,i',Df1,d)
np.einsum('ij,i,j',Df2,d,d)
np.einsum('ijk,i,j,k',Df3,d,d,d)
```

$$\frac{\partial f}{\partial x} = 2x + 2y \implies \frac{\partial f}{\partial x}(1, 2) = 6 \quad (5.163)$$

$$\frac{\partial f}{\partial y} = 2x + 3y^2 \implies \frac{\partial f}{\partial y}(1, 2) = 14. \quad (5.164)$$

Therefore, we obtain

$$D_{x,y}^1 f(1, 2) = \nabla_{x,y} f(1, 2) = \begin{bmatrix} \frac{\partial f}{\partial x}(1, 2) & \frac{\partial f}{\partial y}(1, 2) \end{bmatrix} = \begin{bmatrix} 6 & 14 \end{bmatrix} \in \mathbb{R}^{1 \times 2} \quad (5.165)$$

such that

$$\frac{D_{x,y}^1 f(1, 2)}{1!} \delta = \begin{bmatrix} 6 & 14 \end{bmatrix} \begin{bmatrix} x-1 \\ y-2 \end{bmatrix} = 6(x-1) + 14(y-2). \quad (5.166)$$

Note that $D_{x,y}^1 f(1, 2)\delta$ contains only linear terms, i.e., first-order polynomials.

The second-order partial derivatives are given by

$$\frac{\partial^2 f}{\partial x^2} = 2 \implies \frac{\partial^2 f}{\partial x^2}(1, 2) = 2 \quad (5.167)$$

$$\frac{\partial^2 f}{\partial y^2} = 6y \implies \frac{\partial^2 f}{\partial y^2}(1, 2) = 12 \quad (5.168)$$

$$\frac{\partial^2 f}{\partial y \partial x} = 2 \implies \frac{\partial^2 f}{\partial y \partial x}(1, 2) = 2 \quad (5.169)$$

$$\frac{\partial^2 f}{\partial x \partial y} = 2 \implies \frac{\partial^2 f}{\partial x \partial y}(1, 2) = 2. \quad (5.170)$$

When we collect the second-order partial derivatives, we obtain the Hessian

$$\mathbf{H} = \begin{bmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} \end{bmatrix} = \begin{bmatrix} 2 & 2 \\ 2 & 12 \end{bmatrix}, \quad (5.171)$$

such that

$$\mathbf{H}(1, 2) = \begin{bmatrix} 2 & 2 \\ 2 & 12 \end{bmatrix} \in \mathbb{R}^{2 \times 2}. \quad (5.172)$$

Therefore, the next term of the Taylor-series expansion is given by

$$\frac{D_{x,y}^2 f(1, 2)}{2!} \delta^2 = \frac{1}{2} \delta^\top \mathbf{H}(1, 2) \delta \quad (5.173a)$$

$$= \frac{1}{2} \begin{bmatrix} x-1 & y-2 \end{bmatrix} \begin{bmatrix} 2 & 2 \\ 2 & 12 \end{bmatrix} \begin{bmatrix} x-1 \\ y-2 \end{bmatrix} \quad (5.173b)$$

$$= (x-1)^2 + 2(x-1)(y-2) + 6(y-2)^2. \quad (5.173c)$$

Here, $D_{x,y}^2 f(1, 2)\delta^2$ contains only quadratic terms, i.e., second-order polynomials.

The third-order derivatives are obtained as

$$D_{x,y}^3 f = \begin{bmatrix} \frac{\partial \mathbf{H}}{\partial x} & \frac{\partial \mathbf{H}}{\partial y} \end{bmatrix} \in \mathbb{R}^{2 \times 2 \times 2}, \quad (5.174)$$

$$D_{x,y}^3 f[:, :, 1] = \frac{\partial \mathbf{H}}{\partial x} = \begin{bmatrix} \frac{\partial^3 f}{\partial x^3} & \frac{\partial^3 f}{\partial x^2 \partial y} \\ \frac{\partial^3 f}{\partial x \partial y \partial x} & \frac{\partial^3 f}{\partial x \partial y^2} \end{bmatrix}, \quad (5.175)$$

$$D_{x,y}^3 f[:, :, 2] = \frac{\partial \mathbf{H}}{\partial y} = \begin{bmatrix} \frac{\partial^3 f}{\partial y \partial x^2} & \frac{\partial^3 f}{\partial y \partial x \partial y} \\ \frac{\partial^3 f}{\partial y^2 \partial x} & \frac{\partial^3 f}{\partial y^3} \end{bmatrix}. \quad (5.176)$$

Since most second-order partial derivatives in the Hessian in (5.171) are constant, the only nonzero third-order partial derivative is

$$\frac{\partial^3 f}{\partial y^3} = 6 \implies \frac{\partial^3 f}{\partial y^3}(1, 2) = 6. \quad (5.177)$$

Higher-order derivatives and the mixed derivatives of degree 3 (e.g., $\frac{\partial^3 f}{\partial x^2 \partial y}$) vanish, such that

$$D_{x,y}^3 f[:, :, 1] = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}, \quad D_{x,y}^3 f[:, :, 2] = \begin{bmatrix} 0 & 0 \\ 0 & 6 \end{bmatrix} \quad (5.178)$$

and

$$\frac{D_{x,y}^3 f(1, 2)}{3!} \delta^3 = (y - 2)^3, \quad (5.179)$$

which collects all cubic terms of the Taylor series. Overall, the (exact) Taylor series expansion of f at $(x_0, y_0) = (1, 2)$ is

$$f(x) = f(1, 2) + D_{x,y}^1 f(1, 2) \delta + \frac{D_{x,y}^2 f(1, 2)}{2!} \delta^2 + \frac{D_{x,y}^3 f(1, 2)}{3!} \delta^3 \quad (5.180a)$$

$$\begin{aligned} &= f(1, 2) + \frac{\partial f(1, 2)}{\partial x} (x - 1) + \frac{\partial f(1, 2)}{\partial y} (y - 2) \\ &\quad + \frac{1}{2!} \left(\frac{\partial^2 f(1, 2)}{\partial x^2} (x - 1)^2 + \frac{\partial^2 f(1, 2)}{\partial y^2} (y - 2)^2 \right. \\ &\quad \left. + 2 \frac{\partial^2 f(1, 2)}{\partial x \partial y} (x - 1)(y - 2) \right) + \frac{1}{6} \frac{\partial^3 f(1, 2)}{\partial y^3} (y - 2)^3 \end{aligned} \quad (5.180b)$$

$$\begin{aligned} &= 13 + 6(x - 1) + 14(y - 2) \\ &\quad + (x - 1)^2 + 6(y - 2)^2 + 2(x - 1)(y - 2) + (y - 2)^3. \end{aligned} \quad (5.180c)$$

In this case, we obtained an exact Taylor series expansion of the polynomial in (5.161), i.e., the polynomial in (5.180c) is identical to the original polynomial in (5.161). In this particular example, this result is not surprising since the original function was a third-order polynomial, which we expressed through a linear combination of constant terms, first-order, second-order, and third-order polynomials in (5.180c).

5.9 Further Reading

Further details of matrix differentials, along with a short review of the required linear algebra, can be found in Magnus and Neudecker (2007). Automatic differentiation has had a long history, and we refer to Griewank and Walther (2003), Griewank and Walther (2008), and Elliott (2009) and the references therein.

In machine learning (and other disciplines), we often need to compute expectations, i.e., we need to solve integrals of the form

$$\mathbb{E}_x[f(x)] = \int f(x)p(x)dx. \quad (5.181)$$

Even if $p(x)$ is in a convenient form (e.g., Gaussian), this integral generally cannot be solved analytically. The Taylor series expansion of f is one way of finding an approximate solution: Assuming $p(x) = \mathcal{N}(\mu, \Sigma)$ is Gaussian, then the first-order Taylor series expansion around μ locally linearizes the nonlinear function f . For linear functions, we can compute the mean (and the covariance) exactly if $p(x)$ is Gaussian distributed (see Section 6.5). This property is heavily exploited by the *extended Kalman filter* (Maybeck, 1979) for online state estimation in nonlinear dynamical systems (also called “state-space models”). Other deterministic ways to approximate the integral in (5.181) are the *unscented transform* (Julier and Uhlmann, 1997), which does not require any gradients, or the *Laplace approximation* (MacKay, 2003; Bishop, 2006; Murphy, 2012), which uses a second-order Taylor series expansion (requiring the Hessian) for a local Gaussian approximation of $p(x)$ around its mode.

extended Kalman
filter

unscented transform
Laplace
approximation

Exercises

- 5.1 Compute the derivative $f'(x)$ for

$$f(x) = \log(x^4) \sin(x^3).$$

- 5.2 Compute the derivative $f'(x)$ of the logistic sigmoid

$$f(x) = \frac{1}{1 + \exp(-x)}.$$

- 5.3 Compute the derivative $f'(x)$ of the function

$$f(x) = \exp\left(-\frac{1}{2\sigma^2}(x - \mu)^2\right),$$

where $\mu, \sigma \in \mathbb{R}$ are constants.

- 5.4 Compute the Taylor polynomials T_n , $n = 0, \dots, 5$ of $f(x) = \sin(x) + \cos(x)$ at $x_0 = 0$.

- 5.5 Consider the following functions:

$$f_1(x) = \sin(x_1) \cos(x_2), \quad x \in \mathbb{R}^2$$

$$f_2(x, y) = x^\top y, \quad x, y \in \mathbb{R}^n$$

$$f_3(x) = xx^\top, \quad x \in \mathbb{R}^n$$